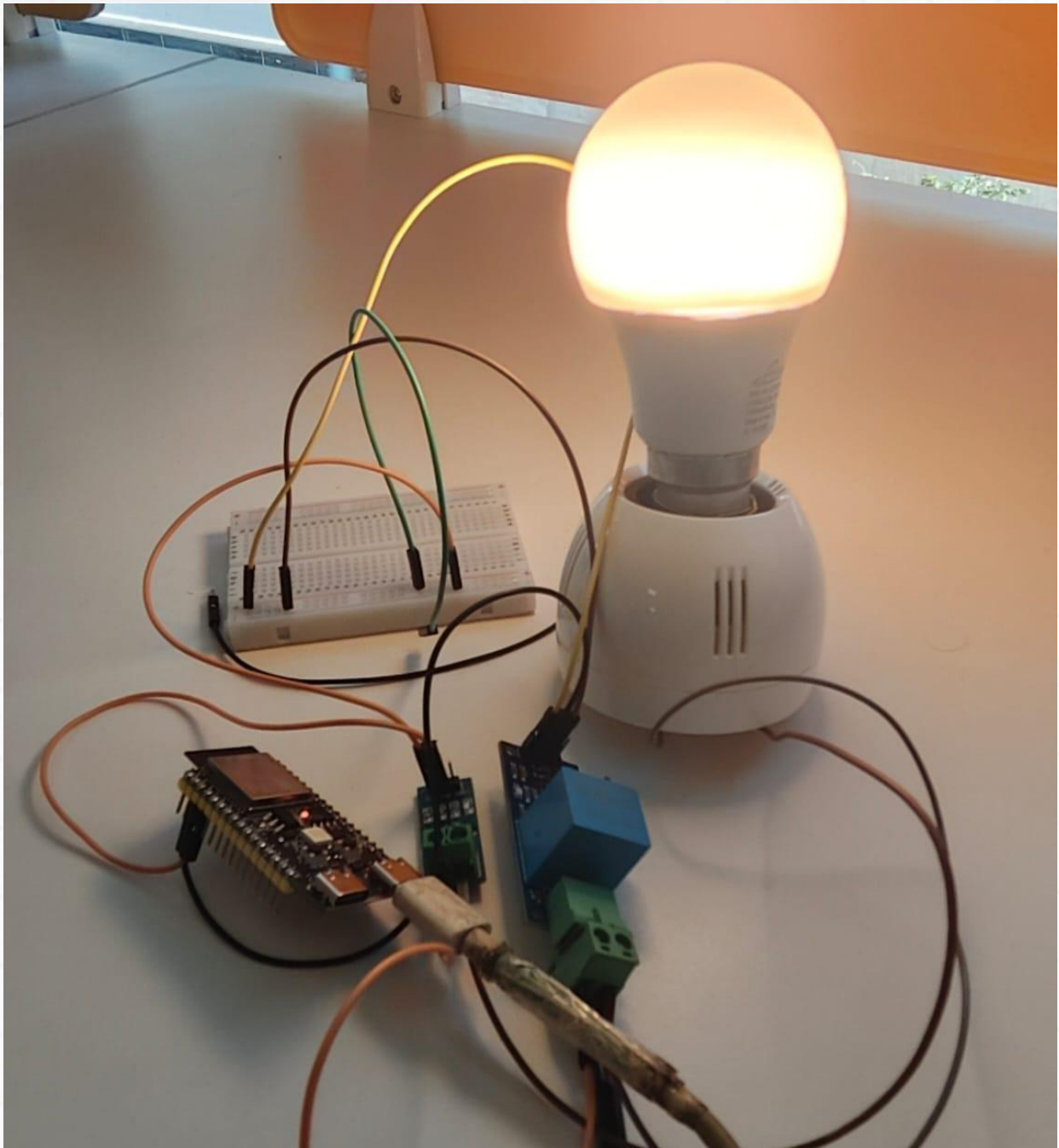


Smart Power (Build an IoT Energy Meter)

(An IoT energy meter that monitors voltage and current, computes power metrics, and displays them on Blynk)



Index

Outcome

Concept

Components

Connections

Software

Tasks

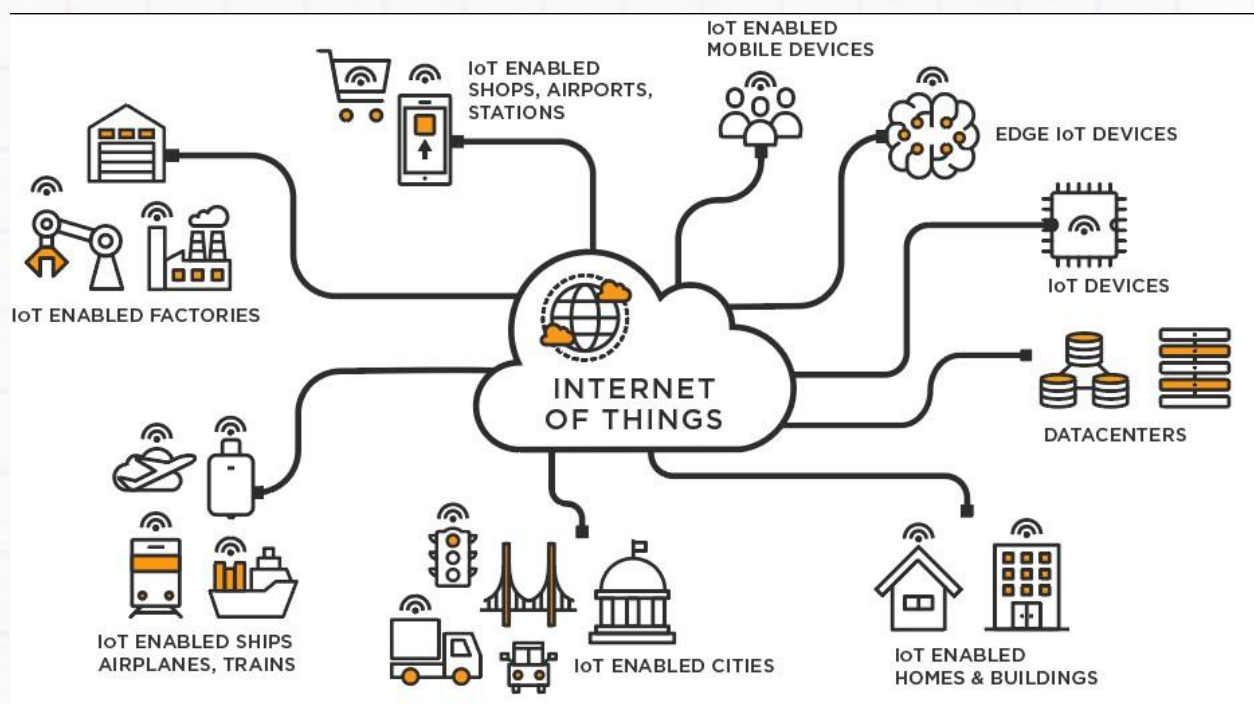
Outcome

Build an IoT (Internet of Things) Energy Meter that measures electrical energy consumption and other parameters in real-time and tracks this data remotely on a web IoT platform.

Concept

What is IoT?

IoT is a system of interconnected computing devices that use internet protocols to communicate and transfer data. This allows remote devices to communicate amongst each other without the need for human involvement,



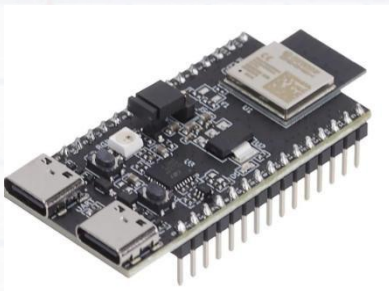
How this project works

- In this project, we use voltage and current sensors and using their values calculate the power and energy consumption from everyday electrical circuits. Since the sensors read the current, voltage values as analog wave signals, which are harder to deal with, we use an Analog to Digital Converter (ADC) to convert them to digital values - i.e. numerical values. These values are received by the ESP 32 C-6 board using the External Interrupt mechanism as we have done for the flow sensor in the IoT Water Meter project.
- We then use these to compute parameters like power factor, power and energy consumed.

- The ESP32C-6 Wi-Fi module sends the data to a blynk cloud server using an API. The energy consumption data can then be viewed in real-time on the blynk web dashboard.

Components

Sno	Components	Quantity
1.	ESP32 C6 Dev Module	1
2.	AC Voltage sensor (ZMPT101B)	1
3.	AC Current sensor (ACS712 20A)	1
4.	60W Bulb	1
5.	Bulb Holder	1
6.	AC 2-pin plug with wiring	1
7.	Jumper wires (Male to Male)	As needed
8.	Jumper wires (Male to Female)	As needed
9.	Bread board	1
10.	USB to type C cable	1



Esp 32 C-6 Module



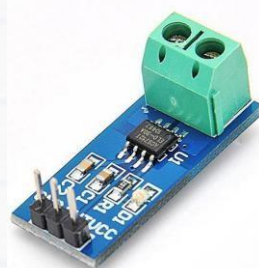
Bulb



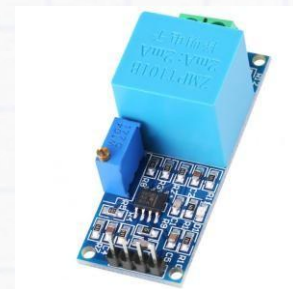
2-pin plug with wiring



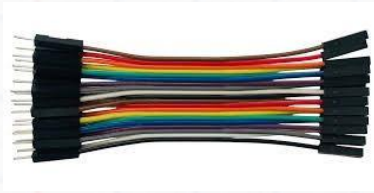
Holder



ACS712 20A Current sensor



ZMPT101B 250V Voltage sensor



Jumper Wires



USB



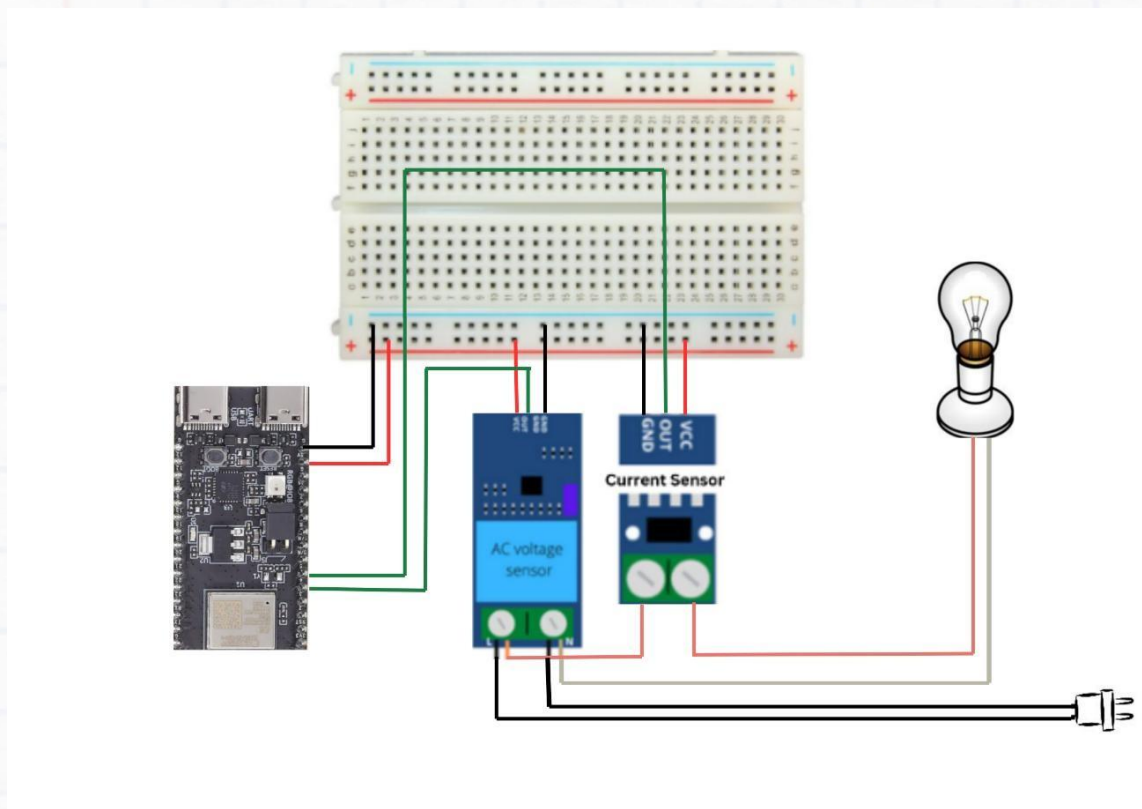
Breadboard

Connections

Caution: High Voltage Electrical Equipment. Do not touch the circuit once it's connected to the power supply. For any changes make sure the power supply is turned off.

Circuit Diagram

NOTE: Before starting the connections, verify that all the jumper wires are working using a Multimeter. Also, ensure that the connections are strong, or the setup may not work.



- VCC to Red line of breadboard
- GND to Blue line of breadboard

AC Voltage Sensor:

- VCC Pin: Connected to the red power line on the breadboard.
- GND Pin: Connected to the blue ground line on the breadboard.
- OUT Pin: Connected to an analog input pin on the ESP32 (GPIO Pin 4).

Current Sensor:

- VCC Pin: Connected to the red power line on the breadboard.
 - GND Pin: Connected to the blue ground line on the breadboard.
 - OUT Pin: Connected to another analog input pin on the ESP32 (GPIO Pin 5).
-
- Connect the AC live and neutral wires to the AC voltage sensor.
 - Pass the AC live wire through the current sensor.
 - Connect the bulb's live wire through the current sensor and the neutral wire directly to the AC power source.

Software

Launching the IDE for our project

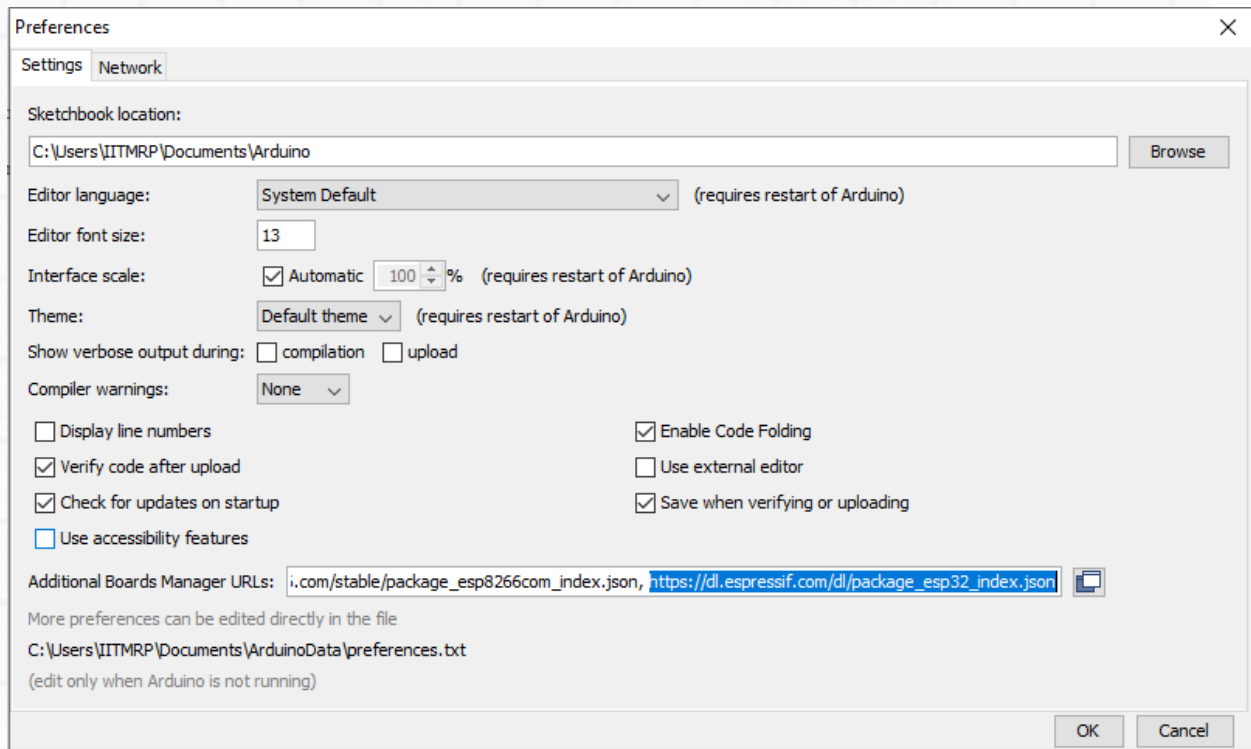
1. Install the Arduino IDE

If you haven't already installed the Arduino IDE, download and install it from the official Arduino website.

2. Install the ESP32 Module in Arduino IDE

- Open the Arduino IDE.
- Go to File > Preferences.
- In the "Additional Board Manager URLs" field, add the following URL:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json



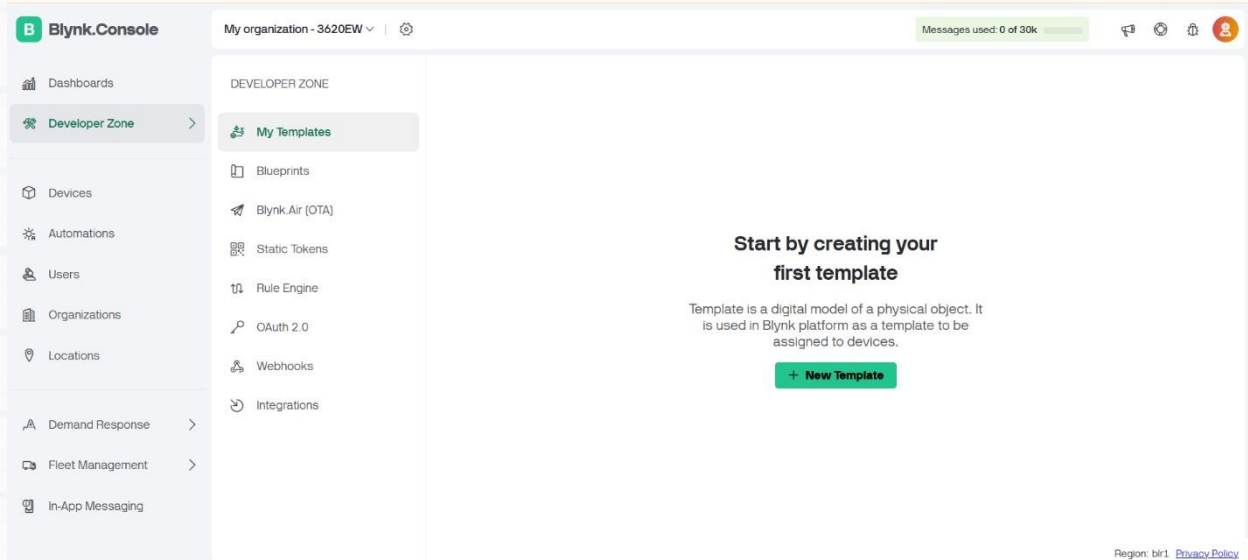
- Go to Tools > Board > Boards Manager.
- Search for ESP32 and install the ESP32 package. This will take some time to install.

3. Install Required Libraries

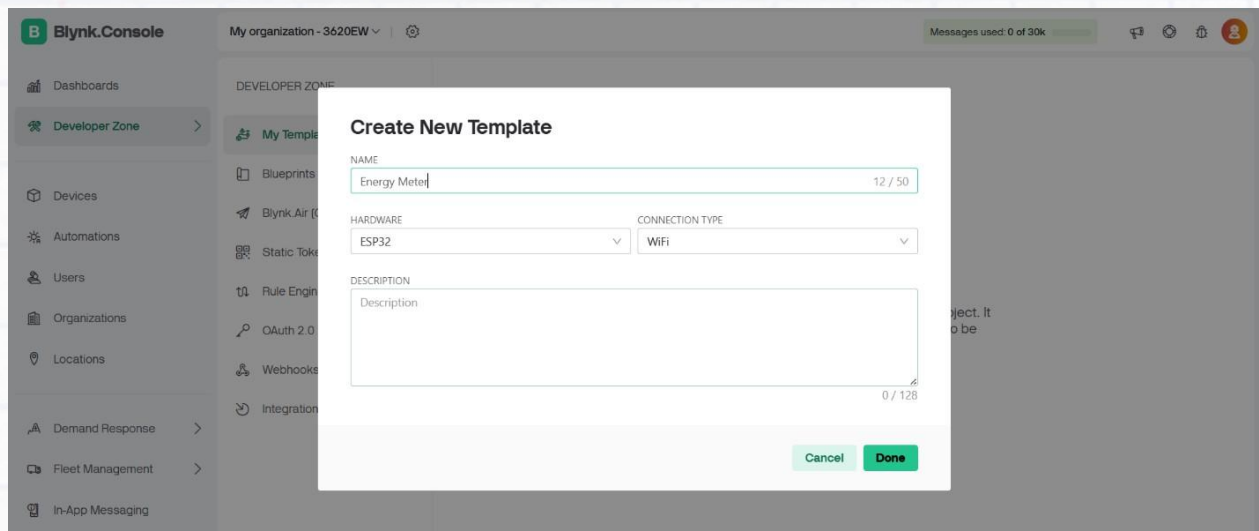
- Go to Sketch > Include Library > Manage Libraries.
- Connect Your ESP32 C-6 Module
- Connect your ESP32 Module to your computer using a USB cable.
- Go to Tools > Board > ESP32C6 Dev Module.
- Go to Tools > Port and select the COM port to which your ESP32C-6 is connected.
- Go to sketch > Include Library > Manage Libraries.
- Search EmonLib and install **EmonLib** by **OpenEnergyMonitor**.

4. Blynk Setup

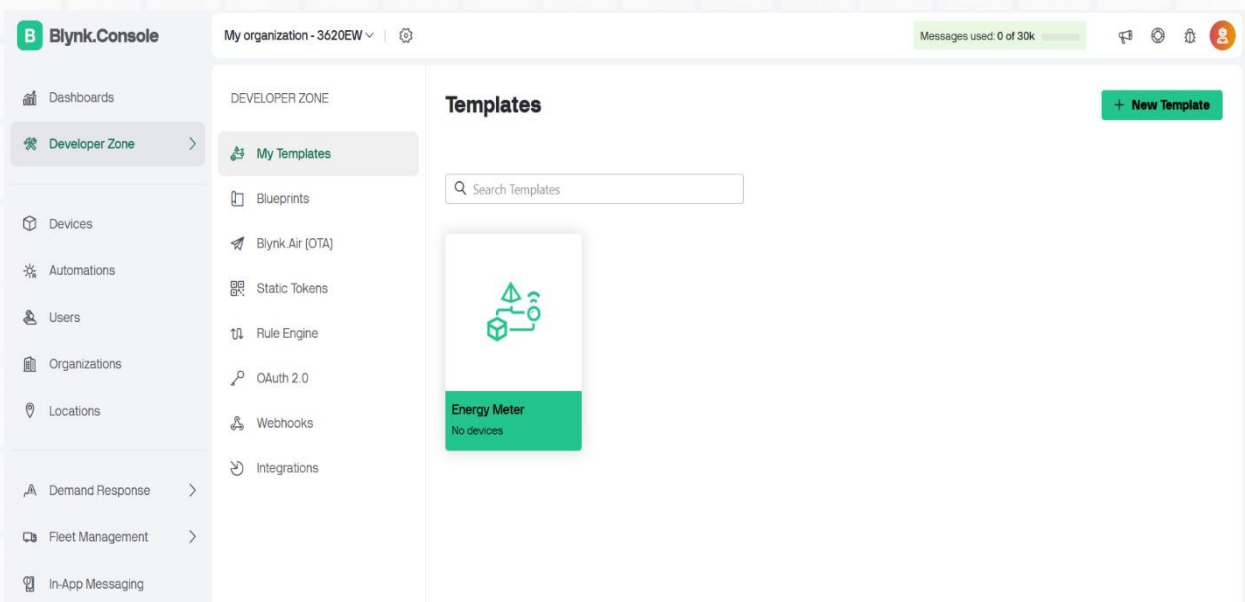
1. For monitoring the values of the electrical parameters, we will be using the Blynk IoT Platform. If you haven't, go to <https://blynk.io/> and sign up for free.
2. On the blynk website, click on Developer Zone >> My templates as shown in the below.



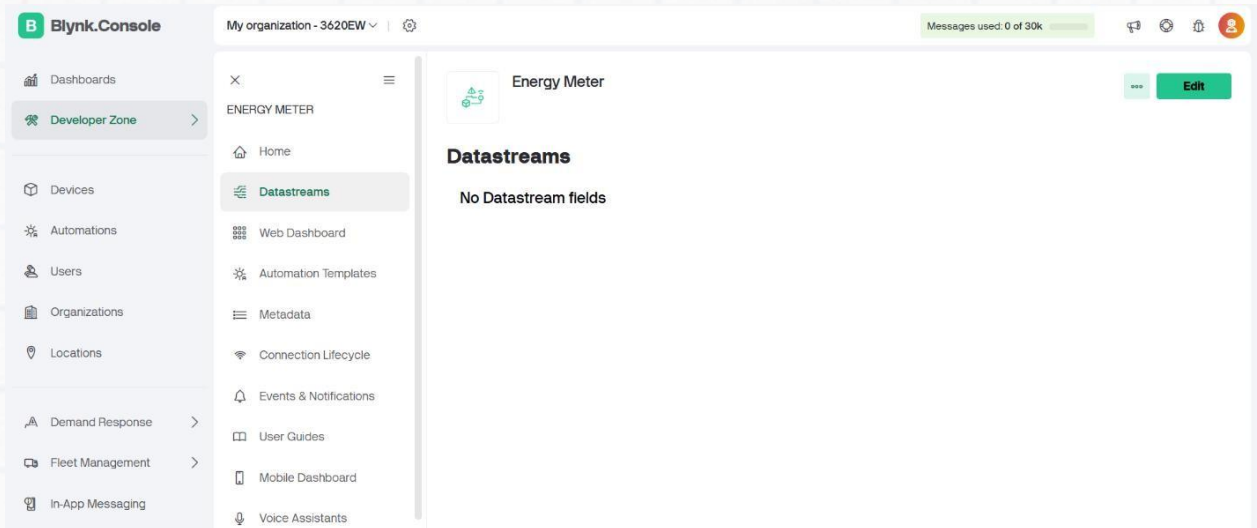
3. Select **New Template** and give the Name of the template as **Energy meter** and check once the hardware is **ESP32** and the connection type is **WIFI** and click done.



4. Now click on Developer Zone >> My templates >> Energy meter.

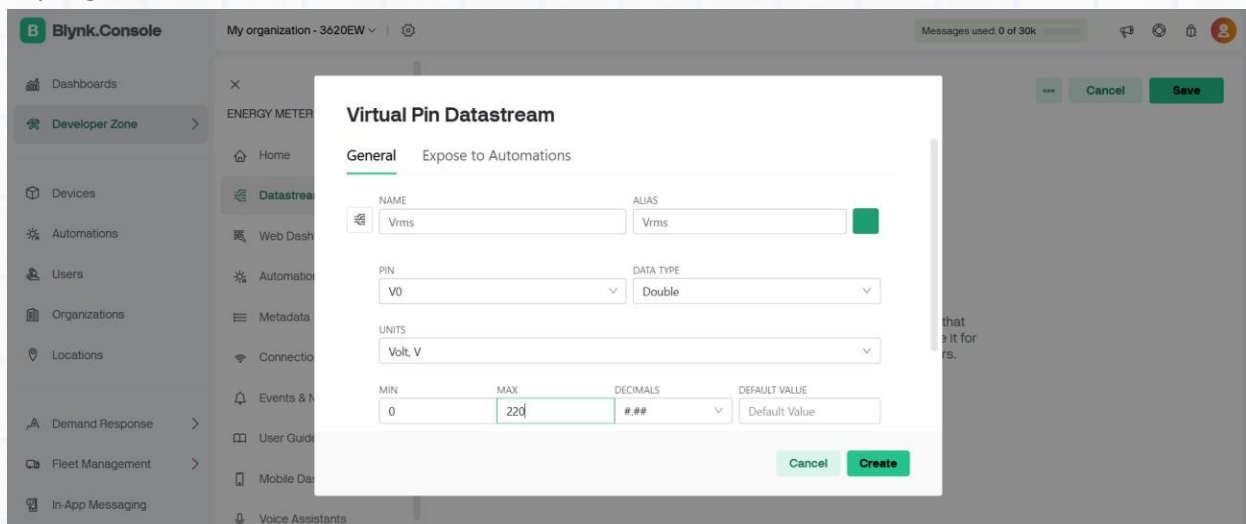


- Now click on Energy meter >> Data streams. After this click on Edit option on the top of the right corner.

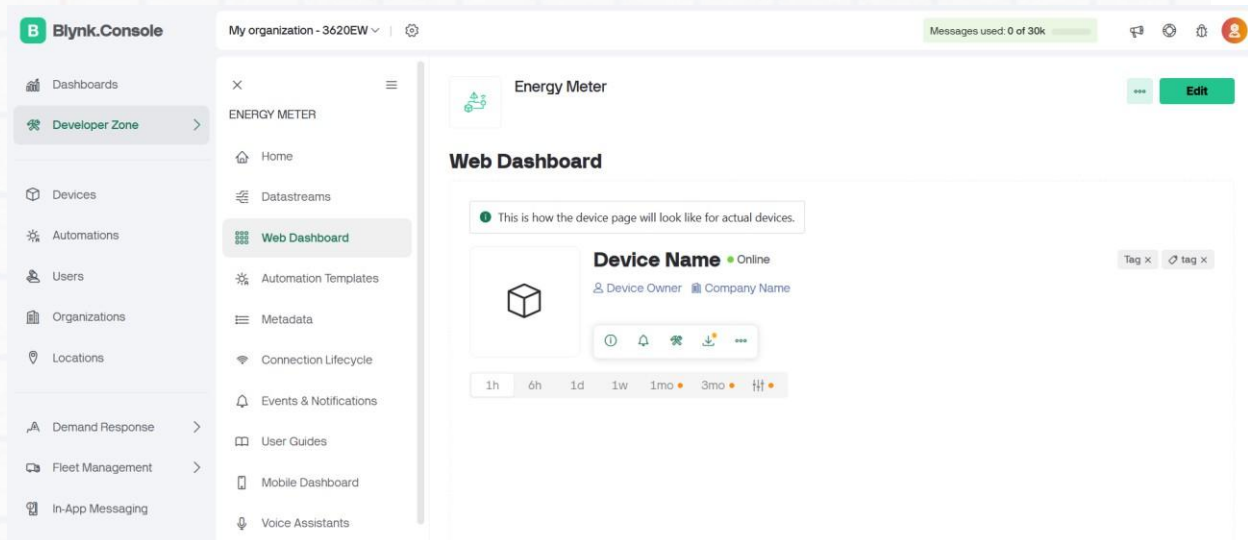


- Now click New DataStream >> virtual pin.
- Give parameters Name as Vrms, Pin as v0, Data type as Double, Units as volt, Min volt as 0 and Maximum volt as 220 and give then click create.

Note: Do the same for remaining parameters like Irms (Max unit 20A), Power(Max unit 500W), Energy(Max unit 300 Wh) and Power factor (Max unit 1) and click save on the top right corner.

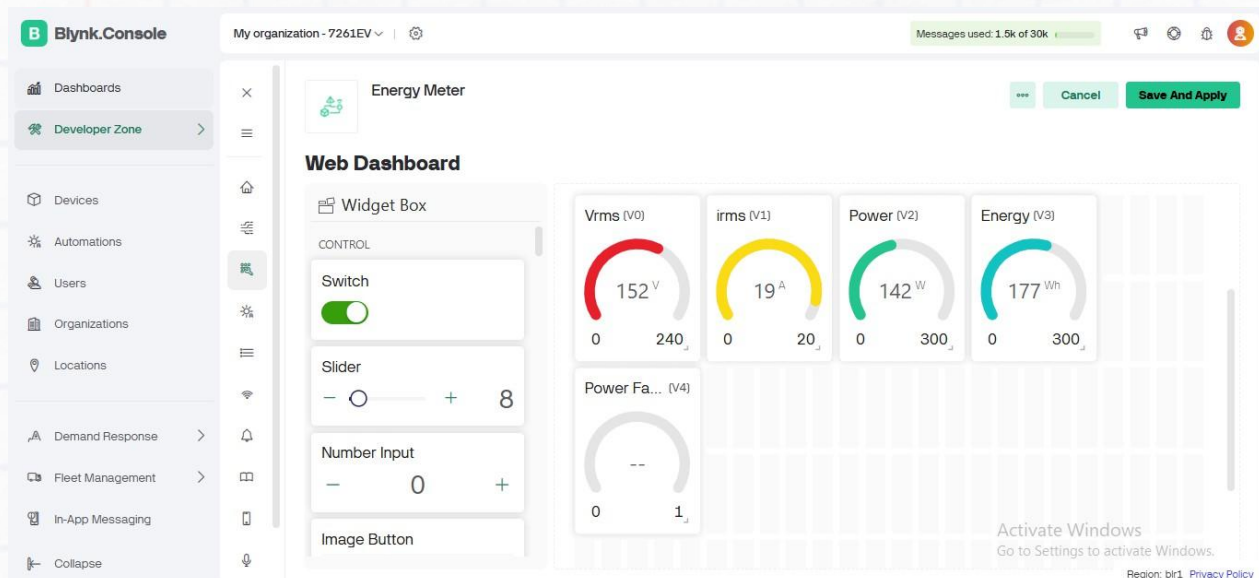


- After that click Developer zone >> My templates >> Energy meter >> Web Dashboard on the left side.



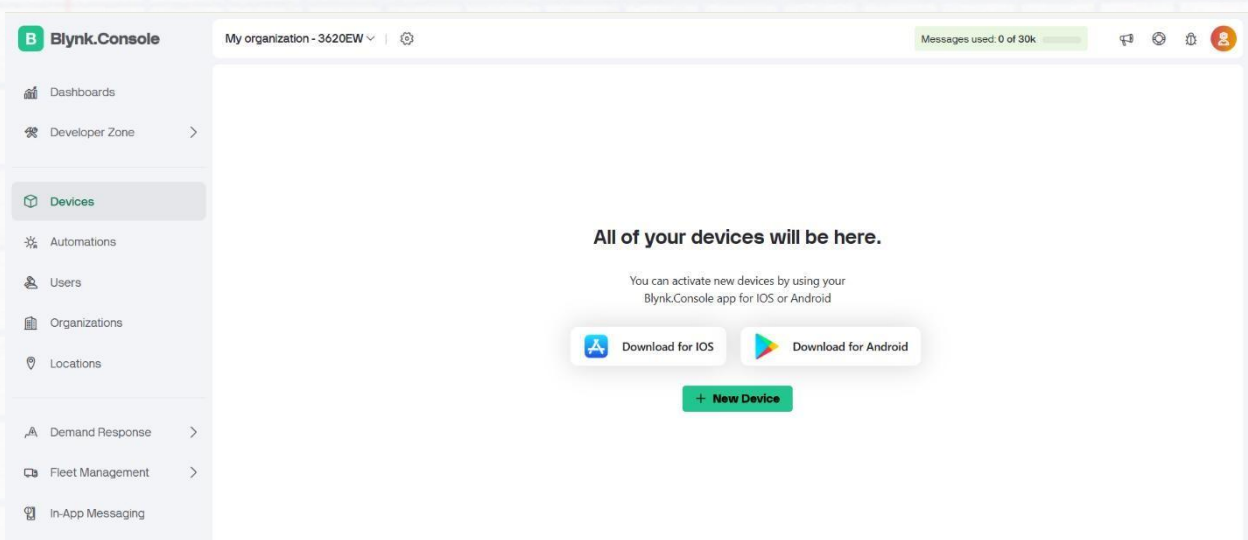
9. Click Edit option in the top of the right side and scroll down the widget box double click Gauge.

10. Click the gear icon (⚙️) in the gauge and give the title as Voltage and data stream as Vrms (V0) and save it.

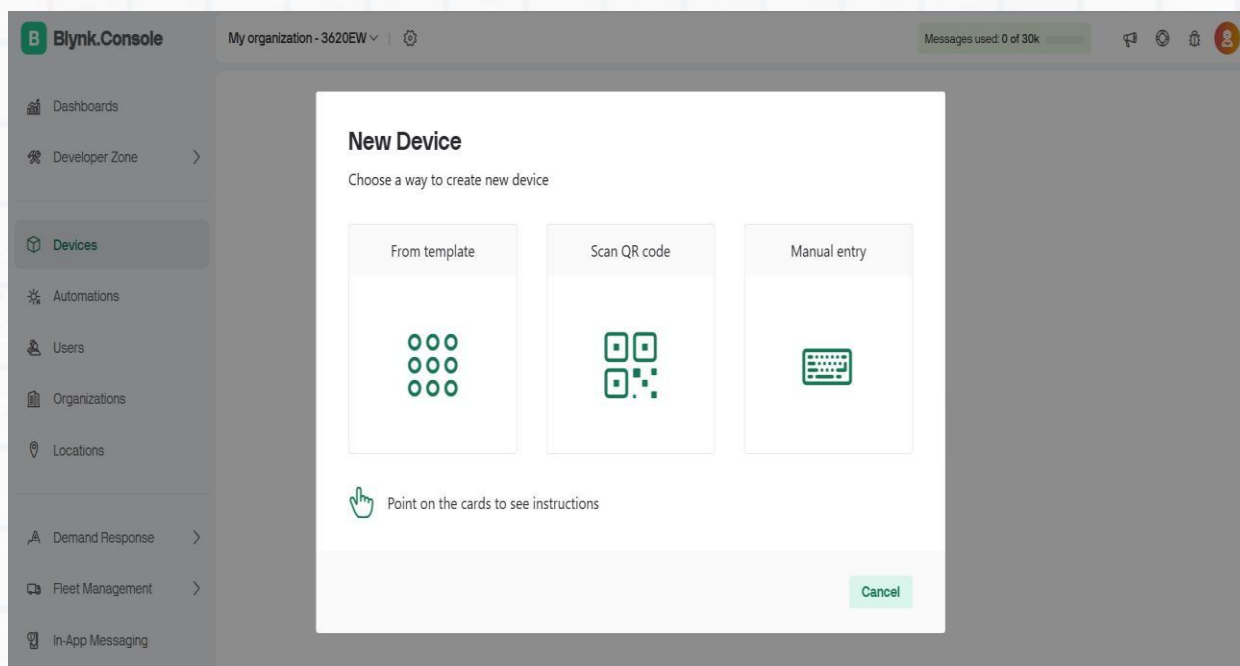


11. Edit the gauges for remaining parameters and Click save and apply.

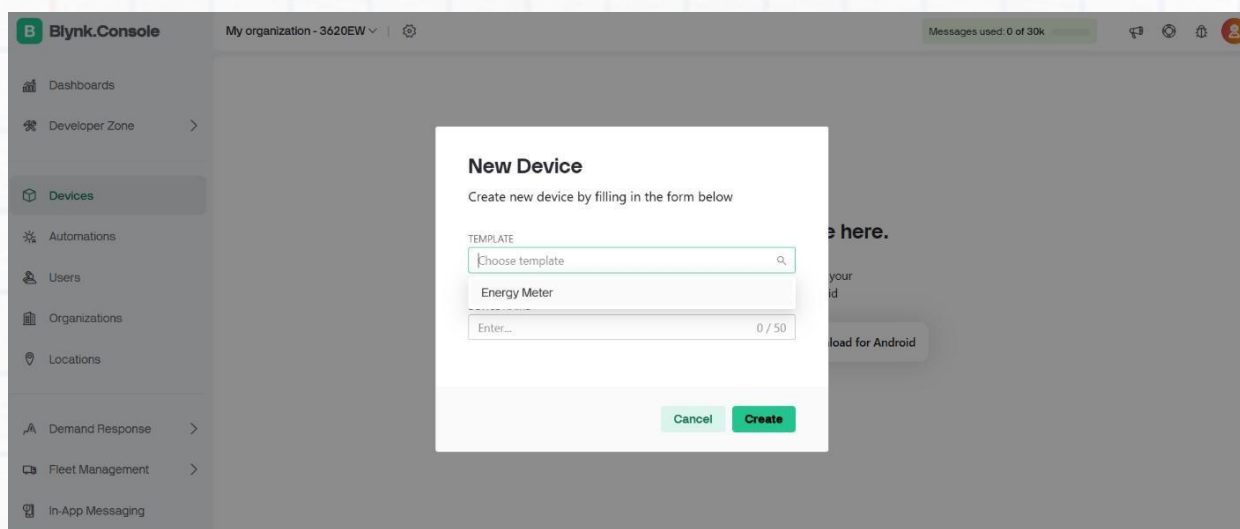
12. Now Click Devices and Click New Device.



13. Click New device and select From Template



14. Now Click Choose device >> Energy Meter and click Create



15. Now everything is done, and you got BLYNK_AUTH_TOKEN Copy that Auth token on the top right part of the page and paste it into the BLYNK_AUTH_TOKEN variable, which is immediately below the Wi-Fi variables in the Arduino code

5. Code

Explanation:

Include Libraries:

- #include "WIFI": Includes the WIFI library to enable Wi-fi connectivity on ESP32.
- #include "EmonLib.h": Includes the Emon Library for energy monitoring
- #include "HTTPClient.h" Includes the HTTPClient library to allow the ESP32 to send HTTP request (Get, Post, etc) to web servers over the internet.

Main code

```
#define BLYNK_TEMPLATE_ID          "TMPL3vnwmtQw0"
#define BLYNK_TEMPLATE_NAME        "IoT energy"
#define BLYNK_AUTH_TOKEN           "06rMGlyd0cbXC0HD1w25816qUV15neAC"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <EmonLib.h>
#include <EEPROM.h>

// Wi-Fi credentials
const char ssid[] = "Mahesh";
const char pass[] = "123456789";

// Energy Monitor
EnergyMonitor emon;
BlynkTimer timer;

float kWh = 0.0;
unsigned long lastMillis = 0;

void setup() {
  Serial.begin(115200);
  delay(100);

  // Connect to Blynk
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

  // Initialize sensors (pins for ESP32-C6)
  // Adjust calibration factors as per your sensor module
  emon.voltage(4, 41.5, 1.7); // Pin 4 for voltage sensor
  emon.current(5, 0.15);      // Pin 5 for current sensor

  // EEPROM for kWh storage
  EEPROM.begin(32);
  EEPROM.get(0, kWh);
  if (isnan(kWh)) kWh = 0.0;

  lastMillis = millis();
}
```

```
// Send data every 5 sec
timer.setInterval(5000L, sendEnergyDataToBlynk);
}

void loop() {
  Blynk.run();
  timer.run();
}

void sendEnergyDataToBlynk() {
  emon.calcVI(20, 2000);

  float Vrms = emon.Vrms;
  float Irms = emon.Irms;
  float realPower = emon.realPower;
  float pf = emon.powerFactor;

  // kWh calculation based on real power
  unsigned long currentMillis = millis();
  kWh += realPower * (currentMillis - lastMillis) / 3600000.0 / 1000.0;
  lastMillis = currentMillis;

  // Serial output
  Serial.printf("Vrms: %.2f | Irms: %.2f | RealPower: %.2f | PowerFactor: %.2f | kWh: %.4f\n", Vrms, Irms, realPower, pf, kWh);

  // Send to Blynk
  Blynk.virtualWrite(V0, Vrms);
  Blynk.virtualWrite(V1, Irms);
  Blynk.virtualWrite(V2, realPower);
  Blynk.virtualWrite(V3, pf);
  Blynk.virtualWrite(V4, kWh);

  EEPROM.put(0, kWh);
  EEPROM.commit();
}
```

After completing all the steps verify the code and upload a code to the ESP32.

6. Upload the Code

- Copy the provided code into a new sketch in the Arduino IDE.
- Fill in your wifi connection's name and password within the empty quotation marks next to the YOUR_WIFI_SSID and YOUR_WIFI_PASS variables. Ensure the Wifi network has no firewalls, else the ESP32 won't connect to it.
- Click the 'Verify' button on the Arduino IDE - shown by a tick symbol. Once you receive the "Done compiling" message, click the 'Upload' button, shown as a

right arrow near the 'Verify' button. Wait until the "Done uploading" message appears.

- After uploading the code, Click the magnifying glass icon on the top-right of the Arduino IDE. It will open the Serial Monitor. If the connection was successful.
- You can see a vrms, irms, power, power factor and energy values.
- Now go to the Devices page of the Blynk website. You will find a device named energy_meter containing variables vrms, irms, power, power factor and energy.



Tasks

- 1) Using a multimeter, calculate the voltage reading of a regular "220V" power socket that you will be using to test the project. Note the reading value.
- 2) Initially, all the values in the Serial Monitor should read zero. Now put the 40W bulb into the bulb holder. Connect the 2-pin plug into the 220V power socket and turn it on.

CAUTION: From now onwards, DO NOT touch any part of the bulb circuit or the sensors while the power is on.

After turning on the bulb circuit, the variables should update in the Serial Monitor. Go to the blynk Dashboard page - the meter gauges should reflect the change in values.

3) Calibration: Is the reading for vrms of the circuit the same as the reading taken using the multimeter before? If not, go to code in arduino IDE and adjust the value of the V_calibration variable until the value of vrms matches the multimeter reading. Since we are using a 40W bulb, the value of power in the Serial Monitor should lie between 39-41 watts. Now that our vrms reading is accurate, inaccuracy in the power value will depend on the irms value's accuracy. This can be fixed by adjusting the I calibration variable in code. Now that calibration is done, we can test our IoT Energy Meter. Turn off the bulb circuit and replace the 40W bulb with the 60W bulb provided in the kit. The value of power should now be between 58-62 watts and the irms value should be greater than before.

Real-world Application

This IoT Energy Meter setup can be used for any 220V single-phase AC appliance (AC here means 'Alternating Current', not 'Air Conditioner'!) that is rated under 5A. This can be done by replacing the bulb with the appliance.

For example, you can replace the bulb with a 220V induction motor and connect a fan to it. For trying out a wider range of appliances, you can replace the bulb holder with a 2-pin female socket. Now, you can directly plug in any permitted appliance (should be rated 220V single-phase AC and under 5A) and monitor its energy consumption.

Make a video presenting how you built and installed the IoT Energy Meter in your college and the various appliances you connected and monitored. Share it with the Build Club Community on the Discord Server!

Club Activities

- 1) Design and execute a similar kind of application to measure the Power across on it.
- 2)